Trabajo Práctico N.º 1

Sistemas Paralelos. Grupo 12

2025

Integrantes:
- Bicocchi Damián 21114/8
- Suarez Francisco 21009/7

Características de Software y Hardware

- Equipo hogareño
  - Sistema operativo: Linux Mint Xia versión 22.1
  - Versión del kernel de Linux: 6.8.0-55-generic
  - Procesador: AMD Ryzen 5 7530U with Radeon Graphics
    - 12 cores
    - 2.00 GHz
    - 384 KiB de caché L1
    - 3 MiB de caché L2
    - 16 MiB de caché L3
  - Memoria RAM
    - 22 GiB
  - Compilador
    - gcc (Ubuntu 13.3.0-6ubuntu2~24.04) 13.3.0
- Clúster remoto (clúster multicore):
  - Clúster conformado por 16 nodos.
  - Cada nodo posee:
    - 8GB de RAM
    - 2 procesadores intel Xeon E5405 de 4 cores a 2.0GHz.

Consideraciones respecto a la compilación y ejecución

La compilación de todos los algoritmos aquí usados se realizó con el nivel de optimización dado por el flag "-O3", que en la práctica fue el que nos dio el segundo mejor resultado. El mejor rendimiento lo tuvimos con la optimización con el flag "-Ofast", pero al poder llevar a errores, nos decantamos por no usarlo.

Enunciado

Punto 1

Resuelva el ejercicio 4 de la Práctica N.º 1 usando dos equipos diferentes: (1) clúster remoto y (2) equipo hogareño al cual tenga acceso con Linux nativo (puede ser una PC de escritorio o una notebook).

Dada la ecuación cuadrática: x2 − 4.0000000 x + 3.9999999 = 0, sus raíces son r1 = 2.000316228 y r2 =1.999683772 (empleando 10 dígitos para la parte decimal).

A. El algoritmo quadratic1.c computa las raíces de esta ecuación empleando los tipos de datos float y double. Compile y ejecute el código. ¿Qué diferencia nota en el resultado?
B. El algoritmo quadratic2.c computa las raíces de esta ecuación, pero en forma repetida. Compile y ejecute el código variando la constante TIMES. ¿Qué diferencia nota en la ejecución?
C. El algoritmo quadratic3.c computa las raíces de esta ecuación, pero en forma repetida. Compile y ejecute el código variando la constante TIMES. ¿Qué diferencia nota en la ejecución? ¿Qué diferencias puede observar en el código con respecto a quadratic2.c? Nota: agregue el flag -lm al momento de compilar. Pruebe con el nivel de optimización que mejor resultado le haya dado en el ejercicio anterior.

(1) Clúster remoto

a. Al ejecutar el algoritmo en el clúster remoto, podemos ver una diferencia no menor entre los dos resultados. En el cálculo utilizando la precisión simple ("*float*"), se nos da como resultado que la parábola solo tiene una raíz, en 2. Obviamente esto no es correcto, y se debe a la poca precisión de este tipo de dato (en C, "*float*" ocupa 4 bytes)
Por otro lado, la solución con Double parece ser exacta a lo que esperábamos. Con la doble precisión nuestro cálculo nos dio las dos raíces que buscamos en la solución del problema. Esto se debe a que este tipo de dato tiene un mayor tamaño de representación (8 bytes en C, el doble que el anterior).

Tipo de dato	Raíz 1	Raíz 2
Float	2.00000	2.00000
Double	2.00032	1.99968

Tabla 1. Raíces computadas con "*quadratic1.c*" en clúster remoto

b. Antes de responder esta pregunta, está bien contextualizar un poco
   i. El código lo que hace es calcular las raíces del punto "*a*" unas 10,000,000 * TIMES de veces
   ii. Como vemos, TIMES es una variable con la que podemos jugar para aumentar o disminuir el número de iteraciones

Ahora sí, podemos ir con los resultados obtenidos por el algoritmo *quadratic2.c*. Los resultados pueden verse en la tabla 2, donde exponemos los tiempos necesarios para cada tipo de dato.

Cantidad de iteraciones (orden de mil millones)	Tiempo requerido con float (En segundos)	Tiempo requerido con double (En segundos)
1	5,987348	6,602799
2,5	14,961680	16,515032
5	29,908125	33,060058
7,5	44,908130	49,611871
10	59,813875	66,566682

Tabla 2. Tiempos del cálculo de "*quadratic2.c*" en cluster remoto con cada tipo de dato

Lo que podemos observar luego de realizar las pruebas, es que a medida que aumentamos la magnitud de las iteraciones, la diferencia entre los tiempos de ejecución del código cuando se usan variables Double y Float persisten (como podemos ver en la tabla 2 tiende a un 10%). Esto nos permite llegar a la conclusión de que las variables de tipo double nos permiten llegar a un resultado con un mayor grado de precisión en su representación, al coste de tener aproximadamente un 10% más de tiempo de cómputo con las especificaciones del clúster remoto.

Debajo mostraremos la ilustración 1 que nos permite tener una visión más clara de la diferencia en tiempos de cómputo entre cada uno de los tipos de datos:

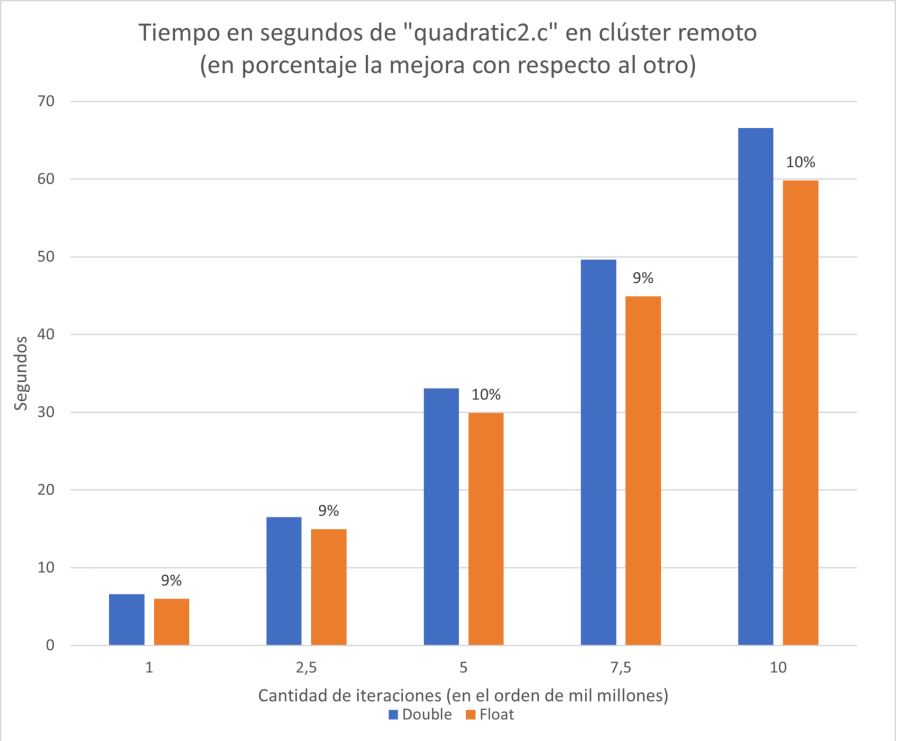

Ilustración 1. Gráfico de barras de los tiempos de cómputo en clúster remoto de "*quadratic2.c*" con cada tipo de dato

c. El algoritmo *quadratic3* tiene la misma idea de calcular las raíces de forma repetida, sin embargo, usa para el tipo de dato *float* funciones específicas para la precisión simple.
Analicemos los resultados que están expuestos en la tabla 3.

Cantidad de iteraciones (orden de mil millones)	Tiempo requerido con float (en segundos)	Tiempo requerido con double (en segundos
1	3,434126	6,555022
2,5	8,615678	16,548462
5	17,061079	33,113933
7,5	25,863997	49,640535
10	34,208525	66,524362

Tabla 3. Tiempos de cómputo en clúster remoto de "quadratic3.c" con cada tipo de dato

Con estas pruebas realizadas en el clúster, podemos ver que la diferencia en tiempo requerido para realizar la ejecución es muy notoria entre la solución que utiliza Double y aquella que utiliza Float, apuntando a una mejora del 50% aproximadamente.

Esta diferencia tan alta puede explicarse por varios factores:

- El coste de las conversiones implícitas cuando se le pasa un “float” a una función que espera un “double” toma tiempo. Al utilizar funciones específicas podemos quitar este “*overhead*”.
- Al tener la mitad del tamaño, no es sorpresa que las mejoras hayan sido de este margen. Muy probablemente los aciertos de caché incrementaron, ya que se pueden almacenar el doble de valores de precisión simple, comparado a los de precisión doble

En la ilustración 2 hay un gráfico de barras para ilustrar las diferencias entre cada tipo de dato en este experimento.

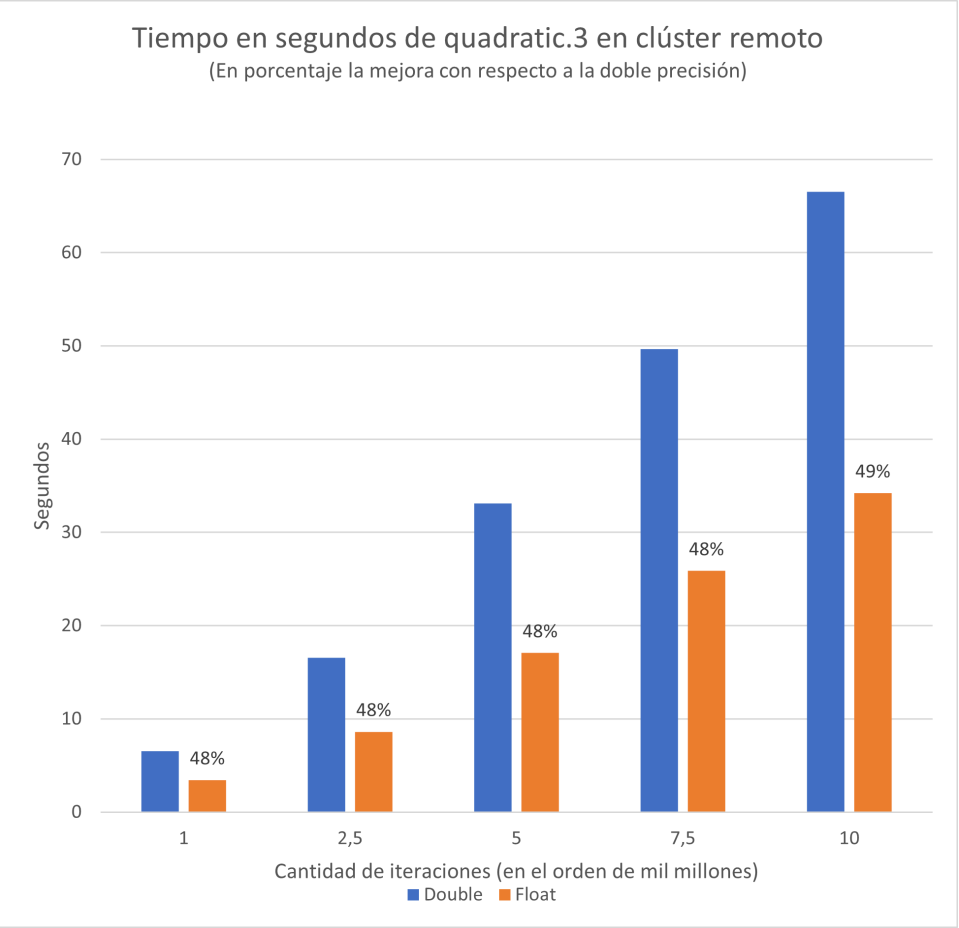

Ilustración 2. Gráfico de barras de los tiempos de cómputo en clúster remoto de “*quadratic3.c”* con cada tipo de dato

(2) Equipo hogareño

A. En la solución con float, se marca que la ecuación solamente tiene una raíz, debido a la poca precisión de este tipo de dato, En la solución con double, podemos ver que se acerca mucho a la solución real, debido a que este tipo de dato tiene un mayor rango de representación, por lo cual se tiene un resultado mucho más preciso.

Tipo de dato	Raíz 1	Raíz 2
Float	2,00000	2,00000
Double	2,00032	1,99968

Tabla 4. Raíces computadas con "*quadratic1.c*" en equipo hogareño

B. TIMES es una variable que determina por cuantas veces multiplicamos 10,000,000, y ese número es la cantidad de veces que recorremos y calculamos las raíces de la ecuación cuadrática. Viendo los resultados obtenidos en el clúster, nos esperábamos encontrar con un resultado similar a los que obtuvimos allí, o incluso nuestra intuición nos indicaba que el tipo de dato double iba a tener un mayor tiempo de cómputo, pero en la práctica esto no se da. Los resultados están plasmados en la tabla 5.

Cantidad de iteraciones (en el orden de mil millones)	Tiempo requerido con float (en segundos)	Tiempo requerido con double (en segundos)
1	0,823682	0,797003
2,5	2,053089	1,956948
5	4,1088	3,881511
7,5	6,168236	5,810936
10	8,231259	8,081947

Tabla 5. Tiempos del cálculo de "*quadratic2.c"* en equipo hogareño con cada tipo de dato

Vemos que la diferencia es ínfima, e incluso da mejores resultados del lado de doble precisión

Algunas explicaciones para esto pueden ser

- El CPU del equipo utiliza instrucciones AVX2: Son un conjunto de instrucciones de 256 bits que se utilizan para operaciones de punto flotante intensivas, utilizando un vector de extensión SIMD. Sabemos que está incluido al haber ejecutado el comando
    - cat /proc/cpuinfo | grep "avx2"

  Y que haya texto donde se indica la presencia del mismo
- La arquitectura Zen3 del procesador pudo haber sido influyente ya que ofrece una menor latencia en operaciones de punto flotante
- Se aprovechó muy bien la caché: La performance de la simple precisión fue mejor en experimentos anteriores ya que la doble precisión puede generar la necesidad de acceder más veces a memoria por un "*miss"* de caché. Los niveles grandes de caché del equipo hogareño deben de ser mayores a los del clúster y quitaron ese acceso adicional

Veamos un gráfico de cómo se comportaron los tiempos de cómputo en cada experimento

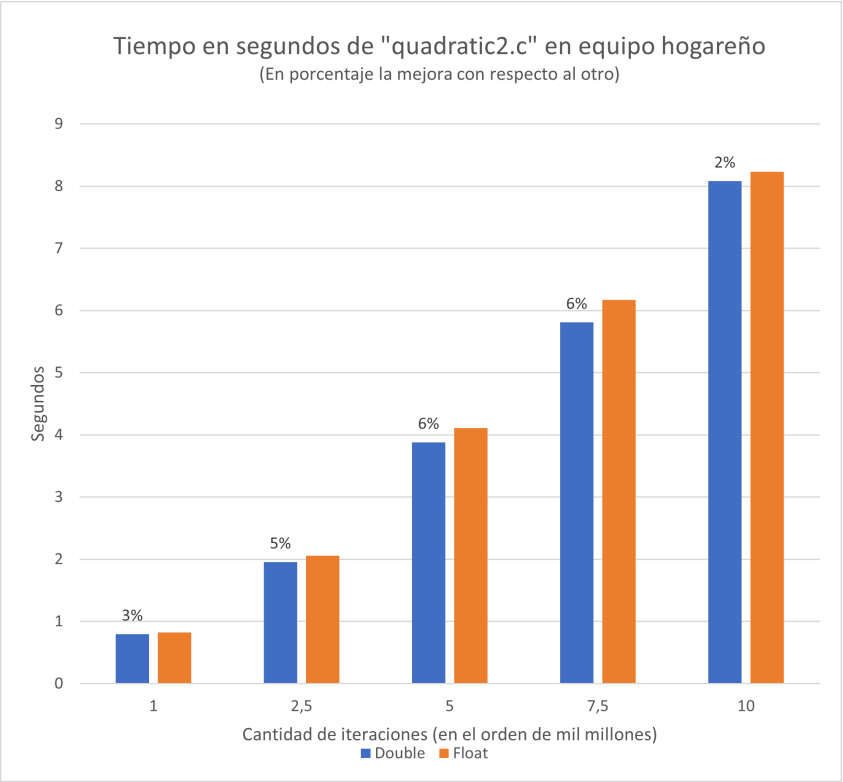

Ilustración 3. Gráfico de barras de los tiempos de cómputo en equipo hogareño de "*quadratic2.c*" con cada tipo de dato

C. La diferencia entre códigos radica en que el cálculo de cada parte se realizan operaciones específicas del tipo en cuestión

- Para los cálculos en precisión simple, por ejemplo
    - El vector se define específicamente de datos float, de manera tal que las operaciones se hagan entre tipos iguales y no se hagan conversiones implícitas
    - Se usan además funciones específicas para float, como "powf" o "sqrtf", lo cual quita el overhead de tratar de convertirlos a double

Podemos ver los resultados en la tabla 6:

Cantidad de iteraciones (en el orden de mil millones)	Tiempo requerido con float (en segundos)	Tiempo requerido con double (en segundos)
1	0,498545	0,800438
2,5	1,262341	1,950802
5	2,566665	3,932893
7,5	3,783935	6,087477
10	4,998952	7,972994

Tabla 6. Tiempos de cómputo en equipo hogareño de "quadratic3.c" con cada tipo de dato

La diferencia positiva es clara con respecto a la precisión simple, que en promedio obtiene una mejora del 36% con respecto a la solución usando precisión doble.

Esta mejora se debe a los mismos factores que en el clúster

- Quitamos la sobrecarga de hacer conversiones implícitas entre datos de simple a doble precisión
- La precisión simple aumenta el número de aciertos caché, ya que “entran” más operandos en la caché

Sin embargo, no encontramos la mejora del 50% en el clúster, probablemente por las razones nombradas anteriormente que influyen en buenos resultados obtenidos con la precisión doble relativos a los obtenidos en el clúster.

- Arquitectura del CPU
- Mayor tasa de aciertos de caché

La ilustración 4 sirve para visualizar mejor los tiempos obtenidos

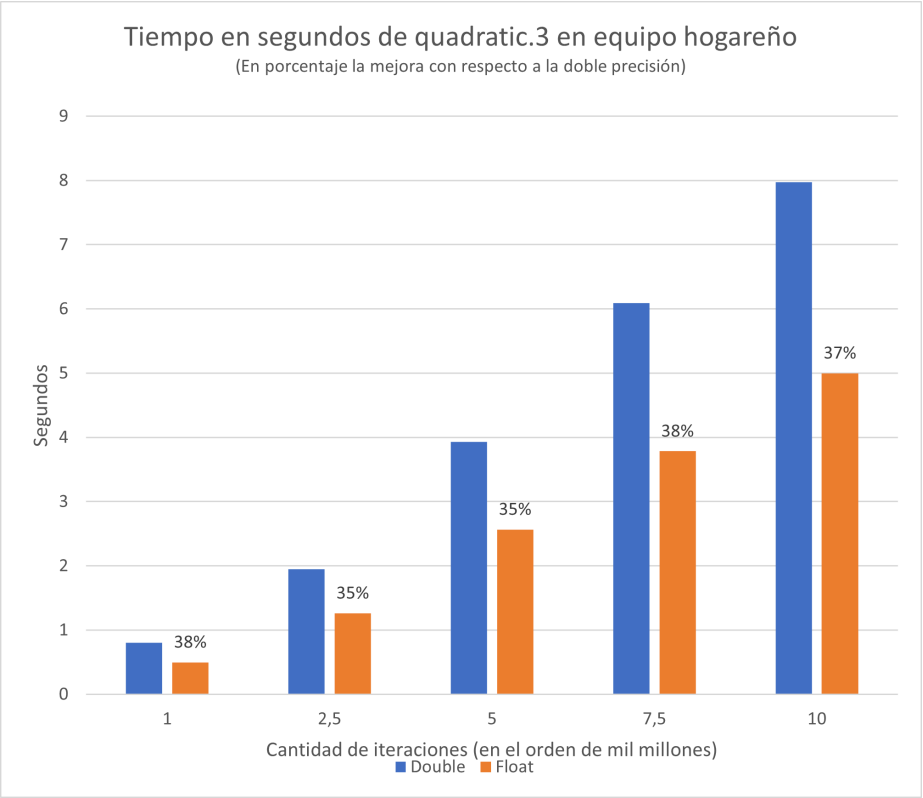

Ilustración 4. Gráfico de barras comparativo de los tiempos netos de “quadratic3.c” en equipo hogareño

Estos experimentos tanto en el clúster como en el equipo hogareño refuerzan la idea de que el tipo de dato float ofrece una mejor performance en cuanto al tiempo de ejecución (y esta diferencia es más notoria si se usan funciones especificadas para dicho tipo de datos) al coste de una representación más acotada.

Por lo tanto, es necesario a la hora de decidir qué tipo de precisión usar, sopesar entre las necesidades de velocidad y precisión.

Enunciado punto 2:

Desarrolle un algoritmo en el lenguaje C que compute la siguiente ecuación:

$$R = \frac{(MaxA \times MaxB - MinA \times MinB)}{(PromA \times PromB)} \times [A \times B] + [C \times B^T]$$

− Donde A, B, C y R son matrices cuadradas de NxN con elementos de tipo "double". − MaxA, MinA y PromA son los valores máximo, mínimo y promedio de la matriz A, respectivamente.

− MaxB, MinB y PromB son los valores máximo, mínimo y promedio de la matriz B, respectivamente.

− B^T Es la matriz transpuesta de B.

Mida el tiempo de ejecución del algoritmo en el clúster remoto. Las pruebas deben considerar la variación del tamaño del problema (N= {512, 1024, 2048, 4096}). Por último, recuerde aplicar las técnicas de programación y optimización vistas en clase.

Respuesta

Antes de mostrar los resultados de la implementación en C de la función pedida en este punto, primero nos gustaría comentar un poco las técnicas de optimización y programación utilizadas en la construcción de este código:

<u>Representación de matrices</u>

En primer lugar, comenzaremos hablando sobre la forma en que representamos las matrices en el código.

Para representarlas lo hicimos por medio de *arreglos dinámicos como vector de elementos*. Esta técnica nos permite

- Utilizar arreglos de una gran longitud, ya que mediante arreglos estáticos corremos riesgo de desbordar la pila solamente con la definición de la estructura
- Almacenar los datos de forma contigua en memoria, lo que nos permite aprovechar la localidad espacial y temporal
- Y nos permiten decidir por qué criterio se van a organizar los datos en la matriz (Por columnas o filas)

```c
void initValorMatrizFila(double *mat, int n, double val)
{
  int i, j;       /* Indexes */
      for (i = 0; i < n; i++)
      {
        for (j = 0; j < n; j++)
        {
          mat[i*n + j] = j;
        }
      }
}
```

Ilustración 5. Captura del código sobre la inicialización de matriz a través de arreglos dinámicos

Organización de las matrices

Para explotar al máximo la localidad espacial, decidimos organizar estratégicamente algunas matrices por filas y otras por columnas. La razón de esta resolución es sencilla: al tener que multiplicar matrices, el multiplicando debe ser “accedido” por filas y el multiplicador por columnas, ya que así se realiza esta operación.

Si ambas matrices estuvieran almacenadas de la misma forma (ambas por filas o ambas por columnas), el acceso a los elementos de alguna de las dos podría ser menos eficiente, ya que estaríamos accediendo a datos no contiguos en memoria. Sin embargo, al almacenar el multiplicando por filas y el multiplicador por columnas, los accesos a los elementos de ambas matrices ocurren en bloques contiguos de memoria. Esto mejora el rendimiento al aprovechar mejor la caché del procesador, reduciendo la cantidad de fallos de accesos a la misma y acelerando los cálculos.

De esta manera, decidimos que en la multiplicación

$$[A \times B]$$

- La matriz *A* se almacena por filas, mientras que *B* se estructure por columnas

Y, por otra parte, en

$$[C \times B^T]$$

- La matriz *C* se almacenaría por filas, mientras que la transposición de *B* sea por columnas

Cálculos de los mínimos, máximos y promedios

Otra técnica que utilizamos para optimizar los tiempos de ejecución para el código en cuestión, es realizar un aprovechamiento de los bucles utilizados para recorrer matrices en busca de un determinado valor. Como en la función dada se pretenden obtener distintos valores (valores máximos, mínimos y promedios), para los cuales se necesita iterar por toda la matriz en busca de estos, lo que hicimos es realizar un solo recorrido por la matriz correspondiente para obtener todos los valores que necesitamos.

Multiplicación por bloques

Siguiendo con las optimizaciones, implementamos la multiplicación de matrices por bloques.

Esta técnica consiste en un método de multiplicación que permite explotar la localidad espacial y temporal maximizando el uso de la memoria caché. En lugar de recorrer fila por fila o columna por columna, lo que hace esta forma de multiplicación es dividir las matrices en submatrices más pequeñas, realizando el producto de estas y luego operar con los resultados parciales

Este método, sin embargo, requeriría de encontrar un tamaño de bloque idóneo para el cual se obtenga el mejor resultado, independientemente de la arquitectura y la caché subyacente en el sistema. En la tabla 7 se muestra el resultado de los experimentos en el clúster

Tamaño de la matriz (N x N)	Tamaño del bloque	Tiempo en segundos en el clúster
512	8	0.611331
	16	0.620747
	32	0,752928
	64	0,728293
	128	0,716463
	256	0,753387
1024	8	8.642908
	16	6.468996
	32	6.877932
	64	6.197770
	128	5.887983
	256	6.255342
	512	7.052020
2048	8	65.782445
	16	47.371815
	32	51.310863
	64	47.048918
	128	46.712461
	256	47.580719
	512	***
4096	8	535.514990
	16	379.504423
	32	442.356640
	64	376.142078
	128	362.706112
	256	***

Tabla 7. Resultados de la experimentación sobre el tamaño de bloque ideal

***: El experimento fue cancelado debido a que tomó una excesiva cantidad de tiempo, se descartó probar valores más grandes en este caso

abc: Mejor resultado con ese tamaño de matriz. (A menor tiempo mejor resultado)

Como podemos observar, los resultados no son del todo consistentes, ya que toman lugar varios factores adicionales, como el tiempo de alocación de memoria o las capacidades de caché de los distintos componentes.

Sin embargo, podemos notar que una zona segura para encontrar los mejores resultados sería un tamaño de bloque de N=128

Variables auxiliares

También, conseguimos mejorar los tiempos utilizando una variable auxiliar a la cual se le va asignando el resultado parcial, minimizando la cantidad de accesos a la matriz resultado. Esto beneficia al rendimiento ya que accedemos menos a memoria para asignar el resultado en lugar de hacerlo repetidamente por cada cambio en este valor, ahorrando accesos a memoria.